Strategy Pattern

Learning Objective

By the end of this sprint, we should be able to:

- Recognize Strategy Pattern opportunities.
- Refactor long if-else chains.
- Add new business behavior without modifying existing code.
- Apply Strategy in real Spring Boot projects.

- Why use Strategy Pattern(មាន if else ច្រើន)

Whenever you see:

```
if (...) {
}

else if (...) {
}

else if (...) {
}
```

AND

each branch contains different behavior,

- Same Goal Difference Algorithm : Strategy Pattern
- Behavor : is the algorithm (ABA, Aclida, …)

យើងបង្កើត Interface ហើយធ្វើការ implementation មួយៗ

Part 3: Extract Behavior

Create interface.

```
public interface NotificationStrategy {

    void send(String message);

}
```

Implement Email.

```
@Component
public class EmailNotificationStrategy
        implements NotificationStrategy {

    @Override
    public void send(String message) {

        System.out.println("Email: " + message);

    }
}
```

Implement SMS.

```
@Component
public class SmsNotificationStrategy
        implements NotificationStrategy {

    @Override
    public void send(String message) {

        System.out.println("SMS: " + message);

    }
}
```

Implement Telegram.

```
@Component
public class TelegramNotificationStrategy
        implements NotificationStrategy {

    @Override
    public void send(String message) {

        System.out.println("Telegram: " + message);

    }
}
```

- Chose strategy : ពេលណាប្រើ SMS, Telegram,…

Answer: We create context.

```
@Service
@RequiredArgsConstructor
public class NotificationService {

    private final Map<String, NotificationStrategy> strategies;

    public void send(String type, String message) {

        NotificationStrategy strategy =
                strategies.get(type);

        if (strategy == null) {
            throw new RuntimeException("Unsupported");
        }

        strategy.send(message);
    }
}
```

Where does Map come from?

Answer: This leads naturally into Spring.

បើយើងចង់យក stragey ណាមួយ យើងប្រើប្រាស់ map.get

Example in spring boot:

```
@Component("EMAIL")
public class EmailNotificationStrategy
        implements NotificationStrategy {

}
```

- Create component and take “email” for the keyword

```
@Component("SMS")
public class SmsNotificationStrategy
        implements NotificationStrategy {

}
```

```
@Component("TELEGRAM")
public class TelegramNotificationStrategy
        implements NotificationStrategy {

}
```

Spring automatically creates:

Map<String, NotificationStrategy>

like:

```
{
    "EMAIL" -> EmailNotificationStrategy,
    "SMS" -> SmsNotificationStrategy,
    "TELEGRAM" -> TelegramNotificationStrategy
}
```

⇨ យក វាដាក់ចូល Map ហើយ យក “component name យកទៅធ្វើជា Key

Part 6: Business Requirement Change

Now business says:

Add Slack

Which class changes?

Answer: None

Create new class only.

```
@Component("SLACK")
public class SlackNotificationStrategy
        implements NotificationStrategy {

    @Override
    public void send(String message) {

        System.out.println(message);

    }
}
```

Part 8: Real Enterprise Examples

Example 1: Payment

PaymentStrategy

Implementations:

ABA
ACLEDA
KHQR
WING